

AutoGrade — Hệ thống chấm điểm code tự động

Vision (Small Project)

Đồ án môn Công nghệ phần mềm cho AI
Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM (HCMUS)

Table of Contents

1. Introduction.....	4
2. Positioning	4
2.1 Problem Statement4	
2.2 Product Position Statement4	
3. User Descriptions	5
3.1 User Profiles5	
3.2 User Task and User Environment6	
3.3 Alternatives and Competition7	
4. Product Features.....	7
4.1 Quản lý tài khoản & xác thực7	
4.2 Quản lý lớp học7	
4.3 Ra đề bài tập có hạn nộp	7
4.4 Nộp bài & chạy thử.....	7
4.5 Chấm điểm tự động theo pipeline 5 mô-đun.....	7
4.6 Rà soát, chỉnh điểm & công bố điểm	7
4.7 Bảng điểm tổng hợp & xuất Excel.....	8
4.8 Bài tập tự luyện tự do.....	8
4.9 Quản lý tài liệu môn học	8
5. Non-Functional Requirements	8
5.1 Yêu cầu về giao diện	8
5.2 Yêu cầu về bảo mật	8
5.3 Yêu cầu về môi trường sử dụng	8

1. Introduction

Mục đích: Tài liệu này thu thập, phân tích và xác định các nhu cầu và tính năng ở mức cao của hệ thống AutoGrade — hệ thống chấm điểm code tự động phục vụ giảng dạy. Tài liệu tập trung vào những khả năng mà các bên liên quan (Giảng viên, Sinh viên) cần và lý do những nhu cầu đó tồn tại. Chi tiết về cách hệ thống đáp ứng các nhu cầu này được trình bày trong tài liệu Đặc tả Use-Case và Software Architecture Document.

Tài liệu tham khảo:

- AutoGrade_UseCase_Specification.docx (v1.0) — đặc tả 14 use case của hệ thống.
- AutoGrade_Software_Architecture_Document.docx (v1.0) — kiến trúc phần mềm của hệ thống.
- README.md, HUONG_DAN_CHAM_DIEM.md — tài liệu kỹ thuật trong repo của đồ án.

2. Positioning

2.1 Problem Statement (Phát biểu bài toán)

The problem of (vấn đề về)	việc chấm bài tập lập trình thủ công tốn nhiều thời gian, thiếu nhất quán và khó phát hiện gian lận
affects (ảnh hưởng đến)	Giảng viên và Sinh viên ở các môn học có bài tập lập trình
the impact of which is (ảnh hưởng là)	Giảng viên phải dành rất nhiều thời gian chấm tay hàng trăm bài mỗi học kỳ, điểm số dễ thiếu nhất quán giữa các bài và khó phát hiện code sao chép giữa Sinh viên; Sinh viên phải chờ lâu để nhận điểm/nhận xét và không có công cụ tự luyện tập với phản hồi tức thời
a successful solution would be (giải pháp thành công sẽ là)	một hệ thống tự động chấm điểm theo rubric nhất quán, tự phát hiện gian lận, đưa nhận xét gần như ngay lập tức, và cho Sinh viên một nền tảng tự luyện tập độc lập với hạn nộp bài

2.2 Product Position Statement (Phát biểu giải pháp)

For (sản phẩm cho ai)	Giảng viên và Sinh viên ở các môn học có bài tập lập trình
Who (làm gì)	cần ra đề, nộp bài và chấm điểm code
The AutoGrade (tên sản phẩm)	Is AutoGrade
That (sẽ)	tự động chấm điểm bài tập code theo rubric riêng của từng Giảng viên (testcase, độ phức tạp, chống gian lận) kèm nhận xét bằng AI, đồng thời cho Sinh viên một nền tảng tự luyện tập có phản hồi ngay
Unlike (khác với)	các LMS chung như Google Classroom/Moodle (chỉ nhận file, không tự chấm code) và các nền tảng luyện tập có sẵn như LeetCode/HackerRank (không gắn với lớp học, hạn nộp và rubric cụ thể của một môn học)

Our product (sản phẩm của chúng tôi)	có thể chấm điểm sát với rubric riêng của từng Giảng viên, tự phát hiện gian lận giữa các bài nộp trong cùng lớp, và chạy được hoàn toàn trên hạ tầng miễn phí mà không cần server riêng
--	--

3. User Descriptions

3.1 User Profiles

Persona 1 — Giảng viên

User type (loại người dùng)	Giảng viên phụ trách các môn lập trình / cấu trúc dữ liệu tại khoa CNTT
Viewpoint (quan điểm)	Muốn dành thời gian cho giảng dạy và tương tác với Sinh viên thay vì ngồi chấm tay hàng trăm bài mỗi tuần; cần đảm bảo công bằng, minh bạch trong chấm điểm
Role/responsibilities (vai trò/nhiệm vụ)	Ra đề bài tập kèm testcase và rubric, theo dõi tiến độ nộp bài, rà soát/điều chỉnh điểm do hệ thống chấm trước khi công bố, xử lý các trường hợp nghi gian lận
Skills (kỹ năng)	Thành thạo ít nhất một ngôn ngữ lập trình (Python/C++/Java) để ra đề và kiểm tra testcase; quen thuộc với khái niệm độ phức tạp thuật toán
Computer / domain experience (kinh nghiệm sử dụng máy tính & nghiệp vụ)	Kinh nghiệm dùng máy tính cao, quen dùng LMS/email/Excel; nhiều năm kinh nghiệm giảng dạy
Frequency of use (tần suất sử dụng)	Theo lịch học kỳ — cao điểm vào các mốc hạn nộp bài tập (vài lần/tuần), thấp hơn giữa kỳ
Age / gender / culture (độ tuổi/giới tính/văn hoá)	Giảng viên đại học Việt Nam, đa dạng, khoảng 28–55 tuổi
Habits (thói quen)	Làm việc trên máy tính cá nhân/laptop tại văn phòng hoặc từ xa; ưu tiên tác vụ thực hiện nhanh trên trình duyệt
Expectations (mong đợi/mong muốn)	Chấm điểm nhanh, công bằng; có thể can thiệp/điều chỉnh khi hệ thống chấm sai; xuất báo cáo điểm dễ dàng
Limitations (giới hạn/hạn chế)	Thời gian hạn chế do dạy nhiều lớp/môn cùng lúc; không phải ai cũng có kỹ năng vận hành server/DevOps

Persona 2 — Sinh viên

User type (loại người dùng)	Sinh viên đại học ngành CNTT, đang học các môn lập trình
Viewpoint (quan điểm)	Muốn biết ngay mình làm đúng/sai ở đâu; muốn tự luyện tập thêm ngoài bài tập bắt buộc để chuẩn bị cho thi cử/tuyển dụng

Role/responsibilities (vai trò/nhiệm vụ)	Tham gia lớp học bằng mã lớp, nộp bài tập trước hạn, xem điểm/nhận xét sau khi công bố, tự luyện tập với các bài tập công khai
Skills (kỹ năng)	Đang học lập trình, mức độ từ mới bắt đầu đến khá; quen thuộc với việc viết và nộp code
Computer / domain experience (kinh nghiệm sử dụng máy tính & nghiệp vụ)	Kinh nghiệm dùng máy tính cao (sinh viên CNTT), quen IDE/editor, git, nộp bài online
Frequency of use (tần suất sử dụng)	Cao vào gần hạn nộp bài tập; có thể dùng thường xuyên hơn nếu tự luyện tập thêm
Age / gender / culture (độ tuổi/giới tính/văn hoá)	Khoảng 18–23 tuổi; phần lớn quen thiết bị di động và laptop
Habits (thói quen)	Thường làm bài tập gần sát hạn nộp (deadline-driven); dùng điện thoại để kiểm tra nhanh nhưng viết code trên laptop
Expectations (mong đợi/mong muốn)	Nộp bài dễ dàng, biết kết quả nhanh, nhận xét cụ thể giúp cải thiện code (không chỉ điểm số)
Limitations (giới hạn/hạn chế)	Đôi khi hạn chế đọc đề bằng tiếng Anh; thời gian hạn chế do học nhiều môn song song

3.2 User Task and User Environment

Tác vụ / mục tiêu	Thực hiện bởi ai, ở đâu	Tần suất	Ràng buộc
Ra đề bài tập kèm rubric/testcase — mục tiêu: chuẩn bị bài tập để Sinh viên nộp	Giảng viên, trên trình duyệt tại nhà/văn phòng	Vài lần mỗi học kỳ (đầu mỗi chương/đợt bài tập)	Cần xác định trước rubric (trọng số testcase/độ phức tạp, ngưỡng gian lận) trước khi Sinh viên nộp bài
Nộp bài tập trước hạn — mục tiêu: hoàn thành yêu cầu bài tập để được chấm điểm	Sinh viên, trên trình duyệt (laptop), thường ở nhà/phòng máy	Theo hạn nộp của từng bài tập, dồn vào gần hạn	Không nộp được sau khi quá hạn; cần đã tham gia lớp học trước đó
Chấm điểm tự động & rà soát trước khi công bố — mục tiêu: có điểm chính xác, công bằng cho cả lớp	Giảng viên, sau khi hết hạn nộp bài	Một lần mỗi bài tập (có thể chấm lại nhiều lần trước khi công bố)	Không thể chấm/sửa lại sau khi đã công bố điểm (khóa vĩnh viễn)
Tự luyện tập với bài tập công khai — mục tiêu: rèn luyện kỹ năng lập trình, chuẩn bị thi cử	Sinh viên, bất kỳ lúc nào, không ràng buộc lớp học	Không cố định, tùy nhu cầu cá nhân	Không có hạn nộp; kết quả chỉ phục vụ cá nhân, không tính vào điểm môn học

3.3 Alternatives and Competition

- Chấm tay truyền thống qua LMS (Google Classroom, Moodle...): quen thuộc, linh hoạt, nhưng chậm, dễ thiếu nhất quán giữa các bài và không tự phát hiện gian lận.
- Các online judge có sẵn (Codeforces, HackerRank, LeetCode, hoặc Themis/DOMjudge dùng cho olympic tin học): mạnh về chạy & so testcase, nhưng không tích hợp quản lý lớp học theo học kỳ, không có rubric tùy biến theo môn, không chấm phong cách bằng AI.
- Tự viết script chấm nội bộ (nhiều giảng viên tự làm): tùy biến cao nhưng tốn công duy trì, thường không có giao diện cho Sinh viên và khó mở rộng.
- Dịch vụ phát hiện đạo văn chuyên biệt (MOSS, Turnitin): chính xác cao nhưng tách biệt khỏi luồng chấm điểm, không tích hợp trực tiếp vào một hệ thống duy nhất, có thể có phí.

4. Product Features

Danh sách các tính năng ở mức cao mà hệ thống AutoGrade cung cấp cho các bên liên quan. Chi tiết luồng xử lý của từng tính năng (Basic Flow, Alternative Flows) được trình bày trong tài liệu Đặc tả Use-Case.

4.1 Quản lý tài khoản & xác thực

Đăng ký, xác nhận email bằng mã 6 số, đăng nhập bằng JWT, quên/đặt lại mật khẩu — dùng chung cho cả Sinh viên và Giảng viên.

4.2 Quản lý lớp học

Giảng viên tạo lớp và nhận mã lớp (join_code) để chia sẻ; Sinh viên tham gia lớp bằng mã đó.

4.3 Ra đề bài tập có hạn nộp

Giảng viên ra đề (văn bản hoặc PDF) kèm rubric tùy biến (trọng số testcase/độ phức tạp, ngưỡng gian lận) và bộ testcase (có thể ẩn).

4.4 Nộp bài & chạy thử

Sinh viên soạn/nộp code ngay trên trình duyệt, có thể chạy thử với testcase công khai trước khi nộp chính thức.

4.5 Chấm điểm tự động theo pipeline 5 mô-đun

Gian lận (gate) → Testcase (sandbox) → Độ phức tạp & Phong cách code (chạy song song, có gọi AI) → Tổng kết điểm và nhận xét bằng AI.

4.6 Rà soát, chỉnh điểm & công bố điểm

Giảng viên xem lại điểm/nhận xét do hệ thống chấm, có thể ghi đề trước khi công bố; sau khi công bố điểm bị khoá vĩnh viễn.

4.7 Bảng điểm tổng hợp & xuất Excel

Giảng viên xem bảng điểm tổng hợp toàn lớp theo từng bài tập và xuất ra file Excel để lưu trữ/báo cáo.

4.8 Bài tập tự luyện tự do

Giảng viên tạo bài luyện tập công khai không gán hạn nộp; Sinh viên làm bất kỳ lúc nào và được chấm điểm ngay khi nộp.

4.9 Quản lý tài liệu môn học

Giảng viên tải lên tài liệu (slide, đề bài PDF...) theo từng mục, Sinh viên trong lớp xem/tải xuống.

5. Non-Functional Requirements

5.1 Yêu cầu về giao diện

- Giao diện web, ngôn ngữ tiếng Việt (giữ thuật ngữ kỹ thuật tiếng Anh khi cần thiết).
- Tối ưu cho màn hình laptop/desktop — tác vụ chính (viết/đọc code) cần bàn phím và màn hình đủ lớn, không ưu tiên bản mobile riêng.
- Soạn code trực tiếp trên trình duyệt bằng Monaco Editor, không cần cài đặt IDE riêng.
- Thông báo lỗi/thành công rõ ràng, tức thời (toast) sau mỗi hành động.

5.2 Yêu cầu về bảo mật

- Mật khẩu người dùng được mã hoá bằng bcrypt, không lưu dạng văn bản thuần.
- Xác thực bằng JWT có thời hạn (mặc định 7 ngày); phải xác nhận email trước khi đăng nhập được.
- Khoá service_role của Supabase chỉ tồn tại ở Backend, không bao giờ lộ ra Frontend.
- Phân quyền rõ theo vai trò: Sinh viên không xem được testcase ẩn hay bài nộp của người khác; Giảng viên chỉ quản lý được lớp/bài tập do chính mình tạo.
- Sandbox chấm code cách ly, giới hạn thời gian và RAM mỗi tiến trình để code không tin cậy của Sinh viên không ảnh hưởng đến máy chủ.

5.3 Yêu cầu về môi trường sử dụng

- Chạy trên trình duyệt hiện đại (Chrome, Edge, Firefox bản gần đây).
- Backend yêu cầu Node.js ≥ 18 ; máy chấm cần cài sẵn toolchain tương ứng ngôn ngữ hỗ trợ (python3, g++, JDK).
- Không yêu cầu tự cài đặt PostgreSQL — dùng Supabase (PostgreSQL as a Service) qua API key.
- Có thể chạy và demo hoàn toàn miễn phí (Supabase free tier, API key miễn phí của Gemini và Groq).